6강. React Router(리우터)

React

리엑트 Router

- **Router**

리액트 라우터(React Router)는 리액트 앱에서 **페이지 이동(Routing)**을 구현하기 위해 가장 널리 사용되는 표준 라이브러리입니다.

리액트는 기본적으로 **SPA(Single Page Application)** 방식이기 때문에, 서버에서 매번 새로운 HTML을 받아오는 대신 하나의 페이지 안에서 필요한 컴포넌트만 갈아끼우는 방식으로 작동합니다.

이때 사용자가 접속한 URL 주소에 따라 어떤 컴포넌트를 보여줄지 결정하는 역할을 리액트 라우터가 담당합니다.

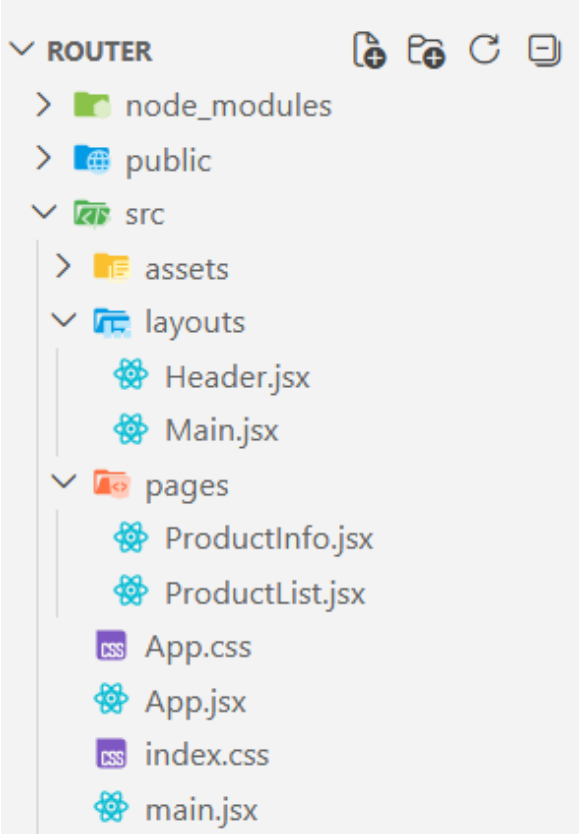
리엑트 Router

- **주요 컴포넌트 및 기능**

구성 요소	설명
BrowserRouter	라우터의 최상위 컴포넌트입니다. HTML5 History API를 사용해 UI와 URL을 동기화합니다.
Routes & Route	경로를 정의합니다. path 속성에는 URL을, element에는 해당 주소에서 보여줄 컴포넌트를 넣습니다.
Link	HTML의 <a> 태그와 비슷하지만, 페이지 새로고침을 막고 라우터를 통해 이동하게 합니다.
useNavigate	특정 이벤트(버튼 클릭 등)가 발생했을 때 프로그래밍 방식으로 페이지를 이동시킬 때 쓰는 Hook입니다.

상품 관리 앱 라우팅

- 상품 관리 프로젝트 구조



상품 관리 앱 라우팅

- **react-router-dom 설치**

```
npm install react-router-dom
```

package.json

```
"dependencies": {
  "react": "^19.2.4",
  "react-dom": "^19.2.4",
  "react-router-dom": "^7.13.1"
},
```

상품 관리 앱 라우팅

- **Header.jsx**

```
import { Link } from "react-router-dom"

const Header = () => {
  return (
    <header className="header">
      <Link to="/">Home</Link>
    </header>
  )
}

export default Header
```

상품 관리 앱 라우팅

- **Main.jsx**

```
const Main = () => {
  return (
    <main>
      <h1>컴퓨터 기기 쇼핑몰</h1>
    </main>
  )
}

export default Main
```

상품 관리 앱 라우팅

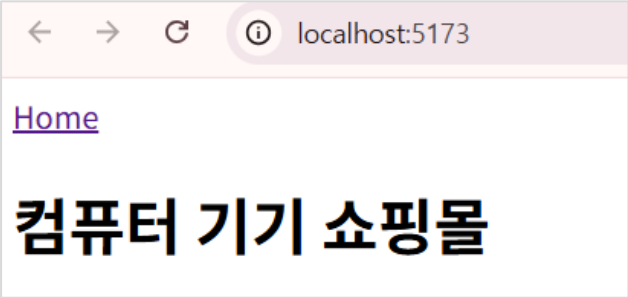
- **App.jsx**

Header 컴포넌트에서 <Link> 컴포넌트를 사용하고 있는데, 이는<BrowserRouter> 내부에서만 작동합니다.
현재 App.jsx에는 라우터 설정이 없어서 에러가 발생하고 화면이 표시되지 않습니다.

```
import { BrowserRouter } from 'react-router-dom'

function App() {

  return (
    <>
      <div className="App">
        <BrowserRouter>
          <Header />
          <Main />
        </BrowserRouter>
      </div>
    </>
  )
}
```

상품 관리 앱 라우팅

- **App.jsx**




```
<body>
  <div id="root">
    <div class="App">
      <header class="header">
        <a href="/" data-discover="true">Home</a> == $0
      </header>
      <main>
        <h1>컴퓨터 기기 쇼핑몰</h1>
      </main>
    </div>
  </div>
  <script type="module" src="/src/main.jsx?t=1773993773892"
</body>
```

Link가 <a>태그로 렌더링 됨

상품 관리 앱 라우팅

- **경로 정의 – Routes, Route**

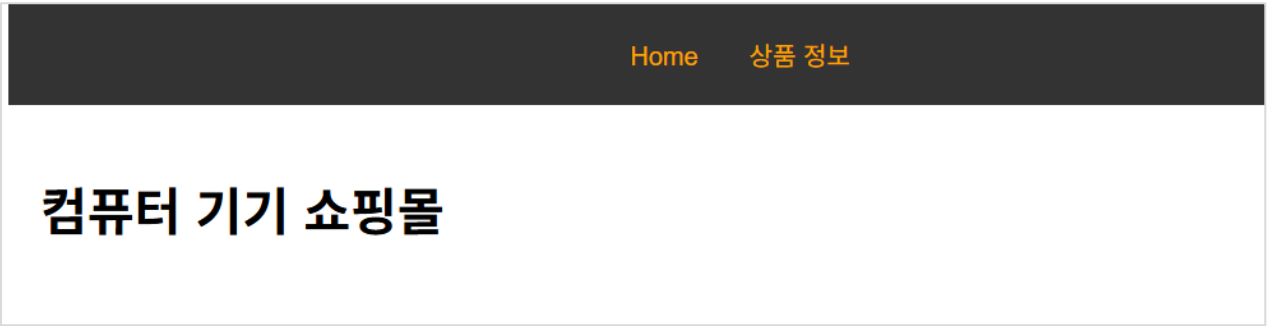
```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
```

```
return (
  <>
    <div className="app">
      <BrowserRouter>
        <Header />
        {/* <Main /> */}
        <Routes>
          <Route path="/" element={<Main />} />
        </Routes>
      </BrowserRouter>
    </div>
  </>
)
```

<Route>는 <Routes> 안에 있어야함

상품 관리 앱 라우팅

- Home 화면



상품 관리 앱 라우팅

- 상품 목록 화면

Home 상품 목록

상품 리스트

모니터

가격: 130000원

설명: 22인치 최신 모니터입니다.

마우스

가격: 30000원

설명: 무선 마우스입니다.

키보드

가격: 50000원

설명: 무선 키보드입니다.

상품 관리 앱 라우팅

- 상품 목록 페이지 이동 – Header.jsx

```
const Header = () => {
  return (
    <header className="header">
      <Link to="/">Home</Link>
      <Link to="/products">상품 목록</Link>
    </header>
  )
}
```

상품 관리 앱 라우팅

- App.css

```
.header{
  background-color: #333;
  padding: 20px;
  text-align: center;
}
.header a {
  margin: 0 16px;
  color: orange;
  text-decoration: none;
}
.header a:hover {
  text-decoration: underline;
}
.main {padding: 20px;}
.product-list, .product-info {padding: 20px;}
```

상품 관리 앱 라우팅

- 객체 데이터 리스트 – ProductList.jsx

```jsx
const ProductList = () => {
  // 객체 리스트
  const products = [
    {
      id: 1,
      name: '모니터',
      price: 130000,
      description: '22인치 최신 모니터입니다.',
    },
    {
      id: 2,
      name: '마우스',
      price: 30000,
      description: '무선 마우스입니다.',
    },
    {
      id: 3,
      name: '키보드',
      price: 50000,
      description: '무선 키보드입니다.',
    },
  ]
```

상품 관리 앱 라우팅

- 객체 데이터 리스트 – ProductList.jsx

```
  return (
    <section className="product-list">
      <h2>상품 리스트</h2>
      {products.map((product) => (
        <div key={product.id}>
          <h3>{product.name}</h3>
          <p>가격: {product.price}원</p>
          <p>설명: {product.description}</p>
        </div>
      ))}
    </section>
  )
}

export default ProductList
```

상품 관리 앱 라우팅

- 상품 상세 보기 – ProductInfo.jsx

```
const ProductInfo = () => {

  return (
    <section className="product-info">
      <h2>상품 정보</h2>
    </section>
  )
}

export default ProductInfo
```

상품 관리 앱 라우팅

- 상품 상세 보기 링크 추가 – ProductList.jsx

```
<h2>상품 리스트</h2>
{products.map((product) => (
  <div key={product.id}>
    <h3>
      <Link to={`/products/${product.id}`}>{product.name}</Link>
    </h3>
    <p>가격: {product.price}원</p>
```

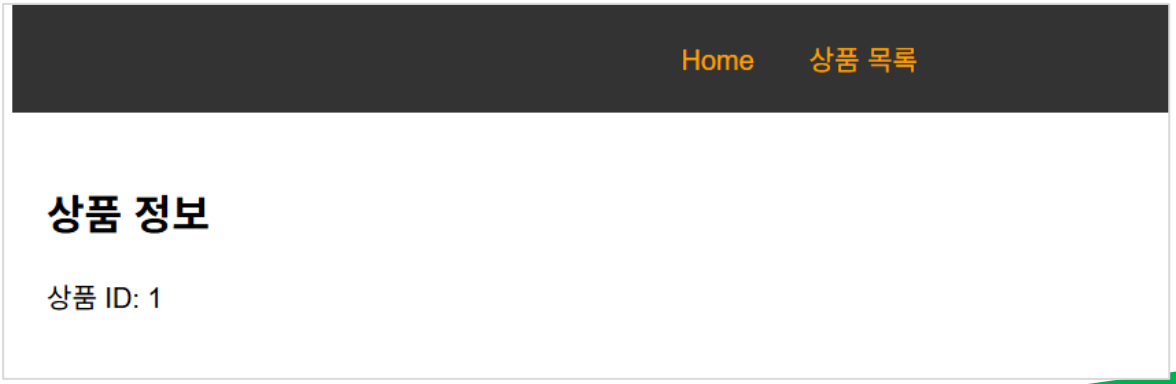
상품 관리 앱 라우팅

- 상품 상세 보기 경로 – App.jsx

```
<BrowserRouter>
  <Header />
  {/* <Main /> */}
  <Routes>
    <Route path="/" element={<Main />} />
    <Route path="/products" element={<ProductList />} />
    {/* URL 파라미터를 포함한 경로 설정 */}
    {/* :id는 URL에서 동적으로 변하는 부분을 나타냅니다. */}
    {/* 예를 들어, /products/1, /products/2 등 다양한 상품 ID에 대응. */}
    <Route path="/products/:id" element={<ProductInfo />} />
  </Routes>
</BrowserRouter>
```

상품 관리 앱 라우팅

- 상품 상세 보기



상품 관리 앱 라우팅

- **useParams()로 id 받아오기**

```
import { useParams } from 'react-router-dom'

const ProductInfo = () => {
  // URL 파라미터에서 id 값을 추출
  // useParams 훅을 사용하여 URL에서 id 값을 가져옵니다.
  // 예를 들어, URL이 /products/1 이면 id는 1이 됩니다.
  const { id } = useParams()

  return (
    <section className="product-info">
      <h2>상품 정보</h2>
      <p>상품 ID: {id}</p>
    </section>
  )
}
```

상품 관리 앱 라우팅

- 데이터 분리하기 – data/product.js

```
export const products = [
  {
    id: 1,
    name: '모니터',
    price: 130000,
    description: '22인치 최신 모니터입니다.',
  },
  {
    id: 2,
    name: '마우스',
    price: 30000,
    description: '무선 마우스입니다.',
  },
  {
    id: 3,
    name: '키보드',
    price: 50000,
    description: '무선 키보드입니다.',
  },
]
```

상품 관리 앱 라우팅

- 데이터 분리하기 – data/product.js

상품 정보

상품 ID: 2

상품 이름: 마우스

상품 가격: 30000

상품 설명: 무선 마우스입니다.

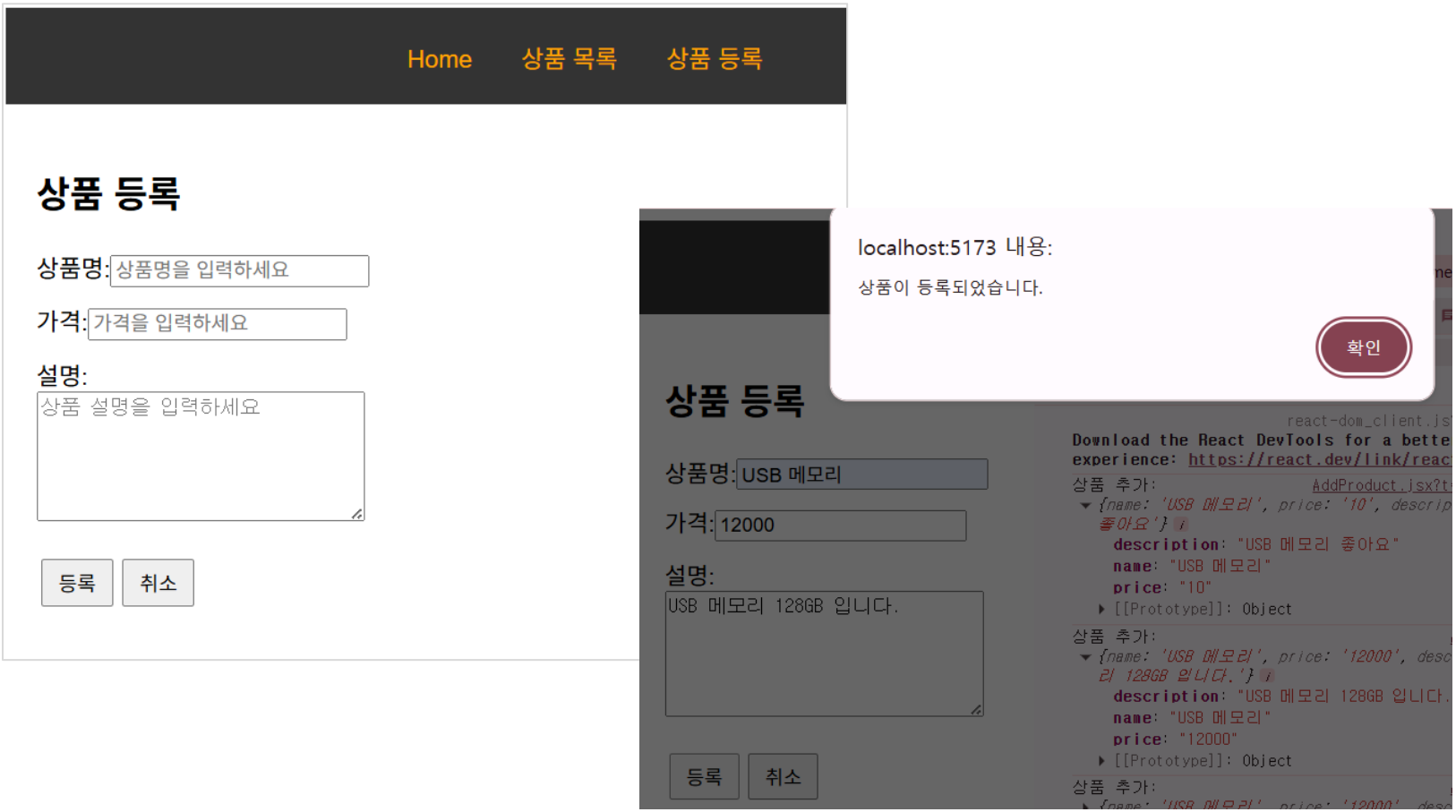
상품 관리 앱 라우팅

- **데이터 분리하기 – data/product.js**

```
import { useParams } from 'react-router-dom'
import { products } from '../data/products'

const ProductInfo = () => {
  // URL 파라미터에서 id 값을 추출
  // useParams 훅을 사용하여 URL에서 id 값을 가져옵니다.
  const { id } = useParams()

  // products 배열에서 id와 일치하는 상품을 찾습니다.
  // products 배열에서 조건에 맞는 첫 번째 요소를 찾습니다
  const product = products.find((p) => p.id === parseInt(id))

  return (
    <section className="product-info">
      <h2>상품 정보</h2>
      <p>상품 ID: {id}</p>
      <p>상품 이름: {product.name}</p>
      <p>상품 가격: {product.price}</p>
      <p>상품 설명: {product.description}</p>
    </section>
```

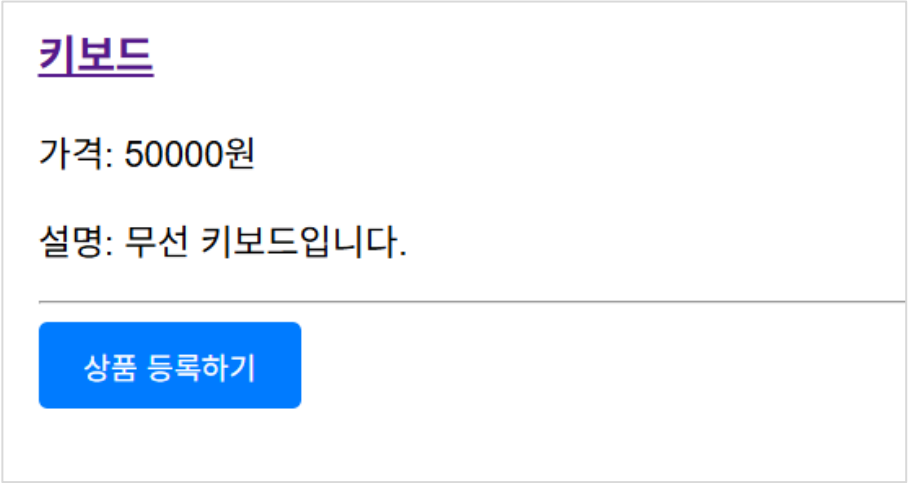
상품 관리 앱 라우팅

- 상품 등록



상품 관리 앱 라우팅

- 상품 목록 페이지에서 등록 버튼 생성



ProductList.jsx

```
  <div className="btn-add">
    <Link to="/add-product">
      <button type="button">상품 등록하기</button>
    </Link>
  </div>
</section>
```

상품 관리 앱 라우팅

- **상품 등록 – AddProduct.jsx**

```
import { useNavigate } from 'react-router-dom'

const AddProduct = () => {
  const navigate = useNavigate() // 페이지 이동을 위한 훅
  const [formData, setFormData] = useState({
    name: '',
    price: '',
    description: ''
  })

  // 입력값 변경 핸들러
  const handleChange = (e) => {
    const { name, value } = e.target
    setFormData({
      ...formData,    // 기존 데이터 복사 (Spread Operator)
      [name]: value,  // 변경된 name 값만 업데이트
    });
  }
```

상품 관리 앱 라우팅

- **상품 등록 – AddProduct.jsx**

```
const handleSubmit = (e) => {
  e.preventDefault()

  // 입력값 검증
  if (!formData.name.trim() || !formData.price ||
      !formData.description.trim()) {
    alert('모든 필드를 입력해주세요.')
    return
  }

  if (isNaN(formData.price) || formData.price <= 0) {
    alert('가격은 0보다 큰 숫자를 입력해주세요.')
    return
  }

  // 상품 추가 처리 (실제로는 API 호출)
  console.log('상품 추가:', formData)
  alert('상품이 등록되었습니다.')
```

상품 관리 앱 라우팅

- **상품 등록 – AddProduct.jsx**

```
    //상품 목록 페이지 이동
    navigate('/products')
}

// 취소 버튼 핸들러
const handleCancel = () => {
    // 폼 초기화
    setFormData({
        name: '',
        price: '',
        description: ''
    })
}
```

상품 관리 앱 라우팅

- **상품 등록 – AddProduct.jsx**

```jsx
return (
  <section className="add-product">
    <h2>상품 등록</h2>
    <form onSubmit={handleSubmit} className='add-form'>
      <div>
        <label htmlFor="name">상품명:</label>
        <input
          type="text"
          id="name"
          name="name"
          value={formData.name}
          onChange={handleChange}
          placeholder="상품명을 입력하세요"
        />
      </div>
      <div>
        <label htmlFor="price">가격:</label>
        <input
          type="number"
          id="price"
          name="price"
          value={formData.price}
          onChange={handleChange}
          placeholder="가격을 입력하세요"
          min="1"
        />
```

상품 관리 앱 라우팅

- **상품 등록 – AddProduct.jsx**

```jsx
    <div>
      <label htmlFor="description">설명:</label>
      <br />
      <textarea
        id="description"
        name="description"
        value={formData.description}
        onChange={handleChange}
        placeholder="상품 설명을 입력하세요"
        rows={5}
        cols={30}
      />
    </div>

    <div className="form-buttons">
      <button type="submit">등록</button>
      <button type="button" onClick={handleCancel}>취소</button>
    </div>
  </form>
</section>
```

상품 관리 앱 라우팅

- 상품 등록 – App.css

```
.btn-add button {
  padding: 10px 20px;
  border-radius: 4px;
  background-color: #007bff;
  color: white;
  border: none;
  cursor: pointer;
}
.btn-add button:hover {
  background-color: #0056b3;
}

/* 상품 등록 폼 */
.add-form div {
  margin-bottom: 10px;
}
.form-buttons button {
  margin: 10px 3px;
  padding: 5px 10px;
}
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – App.jsx

```
function App() {
  // 로그인 상태 관리
  const [isLoggedIn, setIsLoggedIn] = useState(false)
  // 로그인한 사용자 ID 관리
  const [userId, setUserId] = useState('')

  // 로그인 핸들러
  const handleLogin = (userId) => {
    setIsLoggedIn(true) // 로그인 성공 시 상태 업데이트
    setUserId(userId) // 로그인한 사용자 ID 저장
  }

  // 로그아웃 핸들러
  const handleLogout = () => {
    setIsLoggedIn(false)
    setUserId('')
  }
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – App.jsx

```
return (
  <>
    <div className="app">
      <BrowserRouter>
        <Header isLoggedIn={isLoggedIn} userId={userId} onLogout={handleLogout} />
        {/* <Main /> */}
        <Routes>
          <Route path="/" element={<Main />} />
          <Route path="/products" element={<ProductList />} />
          <Route path="/products/:id" element={<ProductInfo />} />
          <Route path="/add-product" element={<AddProduct />} />
          <Route path="/signin" element={<SignIn onLogin={handleLogin} />} />
        </Routes>
      </BrowserRouter>
    </div>
  </>
  )
}
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – Header.jsx

```
import { Link } from "react-router-dom"
import { useNavigate } from "react-router-dom"

const Header = ({ isLoggedIn, userId, onLogout }) => {
  const navigate = useNavigate() // 페이지 이동을 위한 훅

  return (
    <header className="header">
      <Link to="/">Home</Link>
      <Link to="/products">상품 목록</Link>
      <Link to="/add-product">상품 등록</Link>
      {isLoggedIn ? (
        <div className="header-user">
          <span>{userId}님</span>
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – Header.jsx

```
            <button
              type="button"
              className="logout-btn"
              onClick={() => { onLogout(); navigate('/'); }}
            >
              로그아웃
            </button>
          </div>
        ) : (
          <Link to="/signin">로그인</Link>
        )}
      </header>
    )
}
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – App.css

```
.header-user {
  display: inline-flex;
  align-items: center;
  gap: 10px;
  color: #fff;
  margin-left: 12px;
}
.logout-btn {
  padding: 6px 12px;
  border: 1px solid #ff9f1c;
  border-radius: 6px;
  background-color: transparent;
  color: #ff9f1c;
  cursor: pointer;
}
.logout-btn:hover {
  background-color: #ff9f1c;
  color: #333;
}
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
const SignIn = ({ onLogin }) => {
  const navigate = useNavigate(); // 페이지 이동을 위한 훅
  // 1. 입력 데이터를 객체로 통합
  const [loginData, setLoginData] = useState(
    { userId: '', password: '' }
  );
  // 로그인 결과 상태 (성공, 실패, 또는 null)
  const [result, setResult] = useState(null);

  // 2. 통합 핸들러 (name 속성 활용)
  const handleChange = (e) => {
    const { name, value } = e.target;
    console.log(e);

    setLoginData({ ...loginData, [name]: value });
    setResult(null); // 입력 시 결과 메시지 숨김
  };
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
// 3. 제출 핸들러
const handleSubmit = (e) => {
  e.preventDefault();
  const { userId, password } = loginData;

  //find 메서드로 사용자 데이터에서 첫번째 일치하는 항목 찾기
  const matched = users.find(
    (user) => user.userId === userId && user.password === password
  );

  // 로그인 성공 여부에 따라 결과 상태 업데이트
  if (matched) {
    setResult('success');
    onLogin(userId); // App에 로그인 정보 전달
    navigate('/');
  } else {
    setResult('fail');
  }
};
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
return (
  <div className='sign-in'>
    <h2>로그인</h2>
    <form onSubmit={handleSubmit}>
      <p>
        <input
          name="userId" // name 추가
          type="text"
          placeholder="ID 입력"
          value={loginData.userId}
          onChange={handleChange}
        />
      </p>
      <p>
        <input
          name="password" // name 추가
          type="password"
          placeholder="패스워드 입력"
          value={loginData.password}
          onChange={handleChange}
        />
      </p>
```

상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
            <button type="submit">로그인</button>
          </form>

          {/* 결과 메시지 부분 */}
          {result === 'success' && (
            <p style={{ color: 'green' }}>환영합니다, {loginData.userId}님.</p>
          )}
          {result === 'fail' && (
            <p style={{ color: 'red' }}>아이디 또는 비밀번호가 올바르지 않습니다.</p>
          )}
        </div>
      );
    };

    export default SignIn;
```

실습 문제

- **문제**

리엑트 라우터를 이용하여 3개의 페이지를 만들고, 메뉴 클릭시 페이지가 이동하도록 만드세요

Home

About

Board

실습 문제

- **문제**

  동적 라우팅을 구현하세요

```
/board/1
/board/2
```

일반 웹페이지 배포

- **웹 페이지 배포**

1. test web을 깃허브 저장소에 업로드 한다.

2. Settings > Pages 설정하기

   1) **Settings → Pages**
   2) **Build and deployment**:
      - Source: **Deploy from a branch**
      - Branch: **main**를 선택
      - Folder: **/ (root)** 선택
   3) **Save**

일반 웹페이지 배포

- Test 웹

홈페이지 방문을 환영합니다!!

오후 3:17:44

일반 웹페이지 배포

- **index.html**

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Welcome~ Home</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>홈페이지 방문을 환영합니다!!</h1>
    <div id="demo" class="clock"></div>
  </div>

  <script src="clock.js"></script>
</body>
</html>
```

일반 웹페이지 배포

- **style.css**

```
.container {
  display: flex;
  flex-direction: column;
  align-items: center;
}
.clock {
  font-size: 1.5em;
  font-weight: bold;
  color: #00f;
}
```

일반 웹페이지 배포

- **clock.js**

```
setInterval(() => {
  let date = new Date();
  let now = date.toLocaleTimeString();
  document.getElementById("demo").innerText = now;
}, 1000);
```

리엑트 앱 배포

- **Github Pages에 배포 전 사전 작업**

1.Product_mall 앱을 미리 깃허브 저장소에 업로드 한다.

2. Settings > Pages 설정하기

1) **Settings → Pages**
2) **Build and deployment**:
   - Source: **Deploy from a branch**
   - Branch: **main**를 선택
   - Folder: **/ (root)** 선택
3) **Save**

리엑트 앱 배포

- **Github Pages 에 배포**

```
### 1.1 필수 패키지 설치
```bash
npm install
npm install gh-pages --save-dev
```
```

리엑트 앱 배포

- **Github Pages 에 배포**

📝 2단계: package.json 설정

필수 설정:

```json
{
  "name": "todo-app",
  "private": true,
  "version": "0.0.1",
  "type": "module",
  "homepage": "https://username.github.io/repository-name/",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "predeploy": "npm run build",
    "deploy": "gh-pages -d dist",
    "preview": "vite preview"
  },
```

리엑트 앱 배포

- **Github Pages 에 배포**

```
## ⚙️ 3단계: vite.config.js 설정

### 필수 설정:
```javascript
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  base: '/repository-name/',    // GitHub Pages 경로
  server: {
    port: 5173
  }
})
```
```

리엑트 앱 배포

- **Github Pages 에 배포**

```
## 🔗 4단계: index.html 설정

### 필수 설정 (상대 경로 사용):
```html
<!doctype html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>할 일 관리 앱</title>
  </head>
  <body>
    <div id="root"></div>
    <!-- ✅ 상대 경로 사용 (절대 경로 X) -->
    <script type="module" src="./src/main.jsx"></script>
  </body>
</html>
```
```

리엑트 앱 배포

- **Github Pages 에 배포**

```
## 💻 5단계: 로컬 테스트

### 개발 서버 실행:
```bash
npm run dev
```
- http://localhost:5173 에서 접속
- 실시간 핫 리로드 제공

프로덕션 빌드 테스트:
```bash
npm run build
npm run preview
```
- `dist/` 폴더가 생성됨
- 빌드 결과를 테스트할 수 있음
```

리엑트 앱 배포

- **Github Pages 에 배포**

```
## 📤 7단계: 배포하기

### 7.1 배포 명령어:
```bash
npm run deploy
```

7.2 GitHub Pages 활성화:
1. GitHub 저장소 접속
2. **Settings** → **Pages**
3. **Build and deployment**:
   - Source: **Deploy from a branch**
   - Branch: **gh-pages**를 선택
   - Folder: **/ (root)** 선택 ✅ (gh-pages 브랜치의 루트 폴더)
4. Save
```

리엑트 앱 배포

- npm run deploy 후 > Published (배포됨)

```
PS D:\웹 배포\todo-app> npm run deploy

> todo-app2@0.0.1 predeploy
> npm run build


> todo-app2@0.0.1 build
> vite build

vite v4.5.14 building for production...
✓ 34 modules transformed.
dist/index.html                   0.41 kB │ gzip:  0.30 kB
dist/assets/index-d2ef3821.css    1.69 kB │ gzip:  0.72 kB
dist/assets/index-fa09a36a.js   144.01 kB │ gzip: 46.46 kB
✓ built in 880ms

> todo-app2@0.0.1 deploy
> gh-pages -d dist

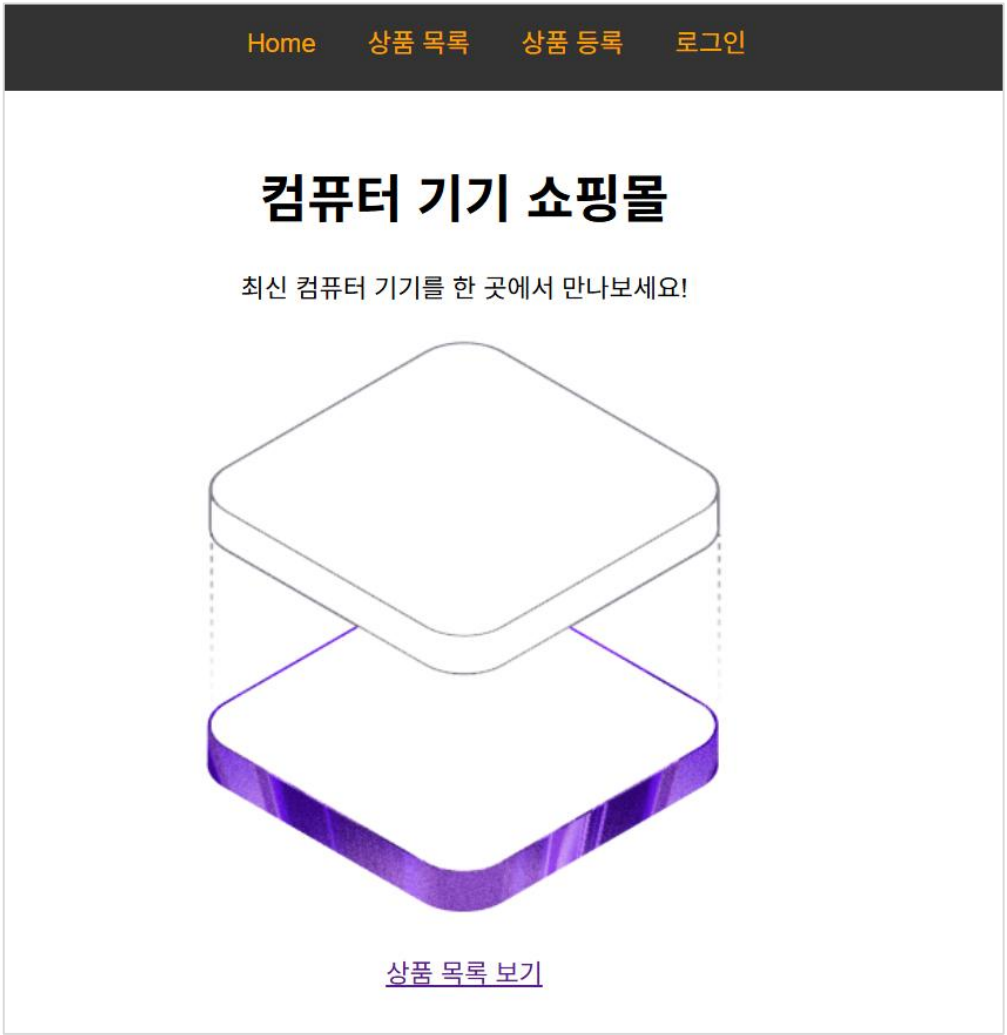
Published
```

업데이트 후 배포

- **Github Pages 에 배포**

```
## 🔄 업데이트 시 배포

코드 수정 후 다시 배포:

```bash
# 1. 로컬 변경사항 커밋 & 푸시 (선택)
git add .
git commit -m "변경사항 설명"
git push origin main

# 2. 배포
npm run deploy
```
```

업데이트 후 배포



업데이트 후 배포

- **Main page 업데이트**

```
const Main = () => {
  return (
    <main className="main">
      <h1>컴퓨터 기기 쇼핑몰</h1>
      <section>
        <p>최신 컴퓨터 기기를 한 곳에서 만나보세요!</p>
        <div>
          <img src={mainImage} alt="메인 이미지" />
        </div>
        <div className="main-link">
          <Link to="/products">상품 목록 보기</Link>
        </div>
      </section>
    </main>
  )
}
```

404 오류 문제

- **404 오류 – File Not Found**

  문제: 새로고침 시 /product_shop/products 같은 경로로 직접 접근하면 GitHub Pages가 해당 파일을 찾으려고 하는데, 실제 파일이 없어서 404가 발생합니다.

  해결방법: public/404.html을 생성하여 index.html과 동일한 내용으로 설정하면, GitHub Pages가 모든 존재하지 않는 경로를 자동으로 index.html로 라우팅합니다.

404 오류 문제

- **404.html – 아직 오류 미해결**

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale
    <title>router</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="./src/main.jsx"></script>
  </body>
</html>
```

문제는 404.html의 스크립트 경로입니다. Vite 빌드 후 404.html은 단순 복사되므로, main.jsx 경로가 무효입니다.

가장 간단한 해결책은 **비트 빌드 후 자동으로** index.html**을** 404.html**로 복사**하도록 설정하는 것입니다:

404 오류 문제

- **vite.config.js 설정 변경**

```
// https://vite.dev/config/
export default defineConfig({
  plugins: [
    react(),
    {
      name: 'copy-404',
      writeBundle() {
        // 빌드 후 dist/index.html을 dist/404.html로 복사
        const distPath = path.resolve(__dirname, 'dist')
        const indexPath = path.join(distPath, 'index.html')
        const notFoundPath = path.join(distPath, '404.html')
        if (fs.existsSync(indexPath)) {
          fs.copyFileSync(indexPath, notFoundPath)
        }
      }
    }
  ],
  base: '/product_shop/'
})
```

추가